Respuestas de las Partes I, II y III

Integración por Monte Carlo en memoria compartida con OpenMP

Parte I. Fundamento del método (15 %)

Notación: el estimador de acierto y fallo usa N puntos (x_i, y_i) uniformes en $[0,1]^2$ y la variable indicadora $X_i = 1$ si $x_i^2 + y_i^2 \le 1$ y $X_i = 0$ si no. Entonces $X_i \sim \text{Bernoulli}(q)$ con $q = \pi/4$ (área del cuarto de círculo) y

$$\hat{\pi}_N = 4\,\frac{1}{N}\sum_{i=1}^{N} X_i = 4\,\frac{\text{dentro}}{N}.$$

I.1. ¿Por qué el error disminuye como $1/\sqrt{N}$?

Intuición. Cada muestra aporta un valor "correcto en promedio" más un error aleatorio. Al sumar N muestras independientes los errores **no se acumulan linealmente**: unos son positivos y otros negativos y se cancelan en parte, como en una caminata aleatoria, cuya distancia típica al origen crece como $\sqrt{N}$. El estimador divide esa suma entre N, así que su error típico es $\sqrt{N}/N = 1/\sqrt{N}$.

Deducción. Si las muestras son independientes e idénticamente distribuidas, la varianza de una suma es la suma de las varianzas:

$$\text{Var}\left(\tfrac{1}{N}\sum_{i=1}^{N} X_i\right) = \frac{1}{N^2}\cdot N\,\text{Var}\,(X_1) = \frac{q(1-q)}{N} \quad\Rightarrow\quad \sigma_{\hat{\pi}} = 4\sqrt{\frac{q(1-q)}{N}} \approx \frac{1{,}642}{\sqrt{N}}.$$

En general, para $I_N = \frac{b-a}{N}\sum f(x_i)$ se obtiene $\sigma_{I_N} = (b-a)\sigma_f/\sqrt{N}$, donde σ_f es la desviación estándar de $f(x)$ con x uniforme: **la constante depende del problema, pero el exponente** $-1/2$ **no**, ni siquiera de la dimensión de la integral.

¿Cuántas muestras para reducir el error a la mitad?

$$\frac{\sigma(N')}{\sigma(N)} = \sqrt{\frac{N}{N'}} = \frac{1}{2} \;\Rightarrow\; N' = 4N.$$

Hacen falta $4N - N =$ **3N muestras adicionales**. Por eso "cuadruplicar las muestras reduce el error a la mitad".

Costo de un dígito de precisión. Ganar un dígito significa dividir el error entre 10: $\sqrt{N'}/\sqrt{N} = 10 \Rightarrow N' = 100N$. El tiempo de cómputo es proporcional a N, así que **cada dígito adicional cuesta 100 veces más trabajo**. Con nuestra versión secuencial ($T(1) = 0{,}8735$ s para $N = 10^8$, ver Parte III), pasar de $\sigma \approx 1{,}6\cdot 10^{-4}$ a $1{,}6\cdot 10^{-5}$ requeriría $N = 10^{10}$ (~87 s), y a $1{,}6\cdot 10^{-6}$, ~2,4 horas. Esta convergencia lenta es la gran debilidad de Monte Carlo en una dimensión; su ventaja es que el exponente no empeora en dimensiones altas, donde las cuadraturas deterministas sí se degradan.

Error típico, error observado y garantía.

Concepto	Qué es	Nuestro caso, $N = 10^8$
Error típico σ	Desviación estándar del estimador: el tamaño *esperado* del error. Es una propiedad de la distribución, no de una ejecución.	$1{,}64\cdot 10^{-4}$
Error absoluto observado	$\lvert\hat{\pi} - \pi\rvert$ en una ejecución concreta. Es *una* realización de una variable aleatoria.	$4{,}07\cdot 10^{-5}$ $(0{,}25\sigma)$
Garantía	No existe cota determinista. Solo hay garantías probabilísticas: por el TCL, $P(\lvert\hat{\pi} - \pi\rvert < 2\sigma) \approx 95\%$; sin suponer normalidad, Chebyshev da $P(\lvert\hat{\pi} - \pi\rvert \ge k\sigma) \le 1/k^2$.	—

Nuestras propias ejecuciones (semilla 20261018) muestran que el error observado **no** baja de forma monótona: con $N = 10^6$ fue $5{,}99\cdot 10^{-4}$ $(0{,}36\sigma)$, con $N = 10^7$ subió a $9{,}39\cdot 10^{-4}$ $(1{,}81\sigma)$ y con $N = 10^8$ bajó a $4{,}07\cdot 10^{-5}$ $(0{,}25\sigma)$. Lo que baja como $1/\sqrt{N}$ es el error *típico*. Además, todo esto supone que el generador pseudoaleatorio se comporta como una secuencia i.i.d., lo cual es una hipótesis sobre el generador, no un hecho matemático.

I.2. ¿Por qué es un problema *embarazosamente paralelo*?

Análisis de dependencias. La iteración i lee solo sus propios números aleatorios, calcula $x_i^2 + y_i^2$ y decide X_i. Ninguna iteración lee un valor producido por otra: **no hay dependencias de datos entre muestras**. Lo único que se acumula es el conteo $\text{dentro} = \sum X_i$, una suma de enteros (asociativa y conmutativa) que puede hacerse por partes y combinarse al final. Por tanto, las N muestras se pueden repartir entre p hilos sin comunicación durante el cálculo: cada hilo procesa su porción y solo al final se suman p números.

Hay dos condiciones para que esto sea correcto:

1. **El generador no debe ser un recurso compartido.** En el código secuencial la variable `estado` de `rand_r` *sí* crea una dependencia entre iteraciones (la iteración i usa el estado que dejó la $i-1$). Esa dependencia es artificial: se elimina dando a cada hilo su propio estado, posicionado en el punto correcto del flujo (Parte II.2).
2. **El acumulador no debe actualizarse concurrentemente sin protección.**

Clasificación de los datos al repartir el muestreo:

Dato	Atributo	Razón
`N`, semilla base, número de bloques	**Compartido** (solo lectura)	Todos los hilos lo leen y nadie lo modifica: no hay carrera.
Índice del ciclo (`b`, `i`)	**Privado**	Cada hilo recorre iteraciones distintas.
`x`, `y`	**Privado**	Valores temporales de cada muestra.
Estado del generador (`estado`)	**Privado**	Si fuera compartido, `rand_r` haría lecturas y escrituras simultáneas sobre él (carrera).
Conteo parcial de aciertos	**Privado**	Cada hilo cuenta sus aciertos sin interferir.
`dentro` total	**Compartido**, pero escrito **solo** mediante la reducción final	Se combina una única vez por hilo.

Riesgos de carrera (comprobados experimentalmente en la Parte II): un estado de generador compartido produce resultados distintos en cada ejecución, y un contador compartido sin protección pierde aciertos.

Utilidad de `reduction(+:dentro)`. OpenMP crea una copia privada de `dentro` en cada hilo, inicializada con el neutro de la suma (0). Cada hilo acumula en su copia sin sincronizarse con nadie, y al terminar el ciclo OpenMP combina las p copias con `+` sobre la variable original. Así el patrón "suma por partes y combina al final" se expresa en una sola cláusula, sin `atomic` ni `critical` dentro del ciclo, y con un costo de combinación $O(p)$, despreciable frente a $O(N)$.

I.3. Equivalencia del estimador secuencial y del paralelo

Demostración. Sea $\{A_1, \ldots, A_p\}$ una **partición** de los índices $\{1, \ldots, N\}$: los conjuntos son disjuntos y su unión es el total. El hilo t procesa los índices de A_t y obtiene $\text{dentro}_t = \sum_{i \in A_t} X_i$. Si cada índice i produce **la misma muestra** (x_i, y_i) que en la versión secuencial, entonces, por la asociatividad y la conmutatividad de la suma:

$$\sum_{t=1}^{p} \text{dentro}_t = \sum_{t=1}^{p} \sum_{i \in A_t} X_i = \sum_{i=1}^{N} X_i = \text{dentro}_{\text{sec}} \quad \Rightarrow \quad \hat{\pi}_{\text{par}} = \frac{4}{N} \sum_t \text{dentro}_t = \hat{\pi}_{\text{sec}}.$$

La igualdad es **exacta**, no aproximada, porque se suman enteros: no hay redondeo y el orden de la suma no importa. Si se acumularan `double`, reordenar la suma podría cambiar los últimos bits.

Que cada índice reciba la misma muestra no es automático: depende del generador. En nuestra implementación se garantiza por construcción, porque cada muestra i usa las llamadas $2i$ y $2i+1$ del flujo de `rand_r` sin importar qué hilo la procese. Se comprobó empíricamente que `dentro` coincide con el secuencial y con el programa del docente para $p = 1, 2, 3, 4, 8, 16$, con `static` y `dynamic` y con tres semillas (`tests/run_tests.sh`, prueba 7).

Precisión con el mismo N total. Aunque cada hilo usara su propio flujo independiente (en lugar de tramos del mismo flujo), el estimador seguiría siendo $\hat{\pi} = \frac{4}{N}\sum_t \text{dentro}_t$ con N variables Bernoulli(q) independientes, de modo que:

$$E[\hat{\pi}] = \pi \ \text{(insesgado)}, \qquad \sigma_{\hat{\pi}} = 4\frac{\sqrt{q(1-q)}}{N} \ \text{(no depende de } p\text{)}.$$

Con el mismo N, la precisión estadística esperada es la misma que con un hilo. El paralelismo reduce el *tiempo* para obtener esas N muestras, no el error. Esto deja de ser cierto si los flujos se solapan: muestras repetidas no aportan información nueva y el N efectivo baja. Por ejemplo, si los p hilos usan la misma semilla, solo hay N/p muestras distintas y σ crece en un factor $\sqrt{p}$ (experimento de la Parte II.3).

Parte II. Implementación en OpenMP (35 %)

Archivos: `montecarlo.c` (versión paralela entregable), `montecarlo_secuencial.c` (línea base), `referencia/montecarlo_docente.c` (copia literal del enunciado), `src/experimentos/` (experimentos controlados). Equipo de medición: `resultados/equipo.txt`.

II.1. Versión secuencial con $N = 10^8$

```
gcc -O2 -fopenmp montecarlo_secuencial.c -o montecarlo_secuencial -lm
./montecarlo_secuencial 100000000
```

Salida (idéntica en las 7 ejecuciones registradas en `resultados/tiempos_speedup.csv`):

```
Estimacion de pi: 3.141633
RESULTADO version=secuencial p=1 ... dentro=78540833 pi=3.1416333200 error=4.070e-05 ...
```

- **Consistencia:** $\hat{\pi} = 3,14163332$; el error absoluto es $4,07 \cdot 10^{-5}$.
- **Origen del orden 10^{-4}:** es el error típico del estimador (Parte I.1), $\sigma = 4\sqrt{q(1-q)/N} = 4\sqrt{0,1686/10^8} = 1,64 \cdot 10^{-4}$. Es decir, con 10^8 muestras se espera un error de algunas diezmilésimas; el nuestro es $0,25\sigma$, totalmente compatible.
- $T(1)$ medido con `omp_get_wtime()` alrededor del ciclo: mediana de 7 ejecuciones **= 0,8735 s** (tiempos individuales: 0,8888; 0,8735; 0,8601; 0,8720; 0,8648; 0,8963; 0,8865 s).

La versión secuencial conserva el programa del docente (mismo `rand_r` de la biblioteca con un único estado y el mismo ciclo). Solo se agregaron la validación de argumentos, la semilla como argumento opcional y la medición de tiempo. `referencia/montecarlo_docente.c` imprime exactamente el mismo valor (prueba 7 de `tests/run_tests.sh`).

II.2. Versión paralela y diseño del generador

Requisitos y cómo se cumplen

Requisito	Implementación en `montecarlo.c`
`omp parallel for` con reducción sobre el acumulador	`#pragma omp parallel for schedule(static) reduction(+:dentro)` (y la variante `dynamic` de II.5)
Elegir p	2.º argumento → `omp_set_num_threads(p)`; si se omite, se usa `OMP_NUM_THREADS`. El programa informa los hilos **realmente** usados (`omp_get_num_threads()`).
Flujo independiente y sin solapamiento por hilo	Cada bloque de muestras crea un estado `rand_r` **privado** posicionado con un salto exacto en el flujo único del generador (ver abajo).
Semilla base asignada	3.er argumento; por defecto `20261018`.
Espaciado de al menos N/p	Con `schedule(static)` el hilo t empieza en la muestra $\approx tN/p$ y consume exactamente su tramo: los tramos son **contiguos y disjuntos**.

Requisito	Implementación en `montecarlo.c`
Reproducible para cualquier p	Cada muestra i recibe siempre los mismos números. Verificado: igualdad **exacta** de `dentro` para $p = 1, 2, 3, 4, 8, 16$, `static` y `dynamic`.

¿Por qué no basta con “semilla distinta por hilo”?

`rand_r(unsigned int *estado)` no tiene función de salto: su único estado es un entero de 32 bits. Revisando la implementación de glibc, cada llamada aplica **tres veces** el generador congruencial lineal (LCG)

$$s_{k+1} = (1103515245\, s_k + 12345) \mod 2^{32}$$

y arma el resultado de 31 bits con bits altos de los tres estados. Como $c = 12345$ es impar y $a - 1$ es múltiplo de 4, el LCG tiene **período completo** 2^{32} (teorema de Hull–Dobell): todos los estados forman un único ciclo. Una semilla no es “un generador distinto”, sino una **posición** en ese ciclo. Dar a cada hilo la semilla $s + t \cdot N/p$ (sumar al *valor* de la semilla) coloca los hilos en posiciones del ciclo que no tienen relación con $t \cdot N/p$: nada impide que el tramo de un hilo alcance el punto de partida de otro.

Lo comprobamos (`src/experimentos/solapamiento.c`, salida en `resultados/solapamiento.txt`). Con la semilla 20261018 y $N = 10^8$, el esquema ingenuo $s + t \cdot N/p$:

p	Tramo por hilo (pasos del LCG)	Esquema ingenuo	Esquema con salto (`montecarlo.c`)
2	300 000 000	sin solapamiento	sin solapamiento
4	150 000 000	sin solapamiento	sin solapamiento
8	75 000 000	**el hilo 1 alcanza la semilla del hilo 0 tras 43 927 968 pasos**	sin solapamiento

En el caso $p = 8$ el desfase (43 927 968) es múltiplo de 6, así que ambos hilos generan **exactamente los mismos 5 178 672 puntos** (31 072 032 pasos / 6), el 5,2 % de N. Que no haya ocurrido con $p = 2$ y $p = 4$ fue cuestión de suerte: si las posiciones fueran aleatorias, el número esperado de solapamientos sería $\approx p(p-1)\,L/2^{32}$ con $L = 6N/p$, es decir 0,14 ($p = 2$), 0,42 ($p = 4$) y 0,98 ($p = 8$). **Semillas diferentes no garantizan ausencia de solapamiento.**

Solución: salto exacto dentro del flujo único

Una transformación afín componida k veces sigue siendo afín: $s_{n+k} = A_k s_n + C_k \pmod{2^{32}}$, con $A_k = a^k$ y $C_k = c\,(a^{k-1} + \ldots + 1)$. Ambos coeficientes se calculan en $O(\log k)$ por duplicación (método de Brown, 1994): función `lcg_saltar`. Como la muestra i consume las llamadas $2i$ y $2i+1$ (6 pasos del LCG), el estado con el que debe empezar la muestra i_0 es

$$s^{(i_0)} = \texttt{lcg_saltar}(\text{semilla},\ 6\,i_0).$$

El ciclo paralelo recorre **bloques** de 65 536 muestras. El bloque que empieza en i_0 salta una sola vez (unas 30 multiplicaciones) y luego llama a `rand_r` normalmente sobre su estado **local**. El estado es una variable automática dentro de `muestrear_bloque`, así que es privado de cada hilo y no se comparte nunca. Resultado:

- **Sin solapamiento, por construcción:** el hilo con las muestras $[i_0, i_1)$ usa exactamente las posiciones $[6i_0, 6i_1)$ del ciclo, y los rangos de muestras son disjuntos. Como $6N \le 2^{32}$, ninguna posición se repite.
- **Espaciado:** con `schedule(static)`, los 1 526 bloques ($N = 10^8$) se reparten en p rangos contiguos. Por ejemplo, con $p = 4$ los hilos empiezan en los bloques 0, 382, 764 y 1 145, es decir, a $\approx N/4$ muestras ($= N/2$ valores de `rand_r`) uno de otro, y cada uno consume justo su tramo.
- **Reproducible para todo p:** el número aleatorio de la muestra i no depende de quién la procese, así que `dentro` es el mismo que el del programa secuencial del docente. No es “muy parecido”: es idéntico (78 540 833 aciertos con $N = 10^8$).

Verificaciones y limitaciones

1. **Dependencia de glibc.** El salto supone que `rand_r` es ese LCG. `montecarlo.c` lo comprueba al iniciar (`rand_r_es_lcg_glibc`: 4 semillas × 1 000 llamadas comparadas con el salto) y se niega a ejecutar si

no se cumple (por ejemplo, en macOS, cuya libc usa otro algoritmo). `tests/test_generador.c` verifica además el salto hasta $2 \cdot 10^6$ llamadas, la aditividad, el período 2^{32} y que cualquier partición en bloques da los mismos aciertos.

2. **Límite de N:** $6N \le 2^{32} \Rightarrow N \le 715\,827\,882$. Por encima, el flujo se repetiría. El programa rechaza valores mayores con un mensaje.
3. **Calidad de `rand_r`.** Los tramos son disjuntos, pero son trozos de **una misma** secuencia LCG de período 2^{32}, con salida de 31 bits. "Independiente" significa aquí *sin solapamiento* y con la misma calidad estadística que el generador del docente; un LCG tiene correlaciones conocidas (estructura en hiperplanos) y no es adecuado para simulaciones exigentes. En este problema el error observado ($0,25\sigma$ con $N = 10^8$) no muestra sesgo, pero eso no demuestra la independencia. Una alternativa más robusta es un generador basado en contador (como la variante SplitMix64 de `src/variantes/`, el diseño original del compañero) o subflujos con salto de generadores modernos (PCG, xoshiro con `jump`). No la usamos como entrega porque el enunciado fija `rand_r` y la semilla del docente, y con esta solución el resultado coincide exactamente con su programa de referencia.
4. **Costo del salto:** 1 526 saltos de ~30 multiplicaciones frente a $2 \cdot 10^8$ llamadas a `rand_r`: despreciable.

II.3. ¿Por qué no compartir el estado global de `rand()`?

`rand()` guarda su estado en una variable **global**. POSIX no exige que sea seguro para hilos. glibc lo protege con un candado, lo que evita la corrupción a costa de serializar todas las llamadas. Hay dos problemas distintos:

(a) Condición de carrera sobre el estado interno. `rand_r(&estado)` hace "leer estado → calcular → escribir estado". Si dos hilos lo ejecutan a la vez sobre el *mismo* `estado`, ambos pueden leer el mismo valor y devolver **el mismo número** (muestras duplicadas), o una escritura pisa a la otra (se pierden avances). Además, qué número recibe cada muestra depende del entrelazado de los hilos, que cambia en cada ejecución. Con un candado (como en `rand()` de glibc) no hay corrupción, pero la asignación de números a muestras sigue siendo no determinista y todos los hilos compiten por el mismo candado.

Experimento (`./defectos estado_global 10000000 4`, `resultados/defectos.txt`), 5 ejecuciones:

Ejecución	dentro	$\hat{\pi}$	tiempo
1	7 857 333	3,142933	0,064 s
2	7 857 464	3,142986	0,318 s
3	7 854 178	3,141671	0,395 s
4	7 858 533	3,143413	0,069 s
5	7 856 768	3,142707	0,401 s
Correcto (5 de 5)	7 856 330	3,142532	0,022–0,029 s

Consecuencias: (i) el resultado no es reproducible; (ii) como el reparto de números depende de los hilos, las muestras dejan de ser una secuencia i.i.d. bien definida, y el error reportado con $\sigma = 4\sqrt{q(1-q)/N}$ no tiene base (las duplicaciones reducen el N efectivo); (iii) el programa es de **2 a 18 veces más lento** que la versión correcta, porque la línea de caché que contiene `estado` rebota entre núcleos en cada llamada. Las mediciones de speedup quedarían invalidadas.

(b) Correlación entre secuencias si todos los hilos usan la misma semilla. Sin carrera (cada hilo con su propio estado), pero inicializados igual, los p hilos generan **la misma secuencia**. Experimento (`./defectos misma_semilla 10000000 4`), 5 de 5 ejecuciones: `dentro = 7 858 232`, exactamente $4\times$ el resultado del secuencial con $N/4 = 2,5 \cdot 10^6$ muestras (1 964 558). **Consecuencias:** el resultado es reproducible (lo que puede engañar) pero solo contiene N/p muestras distintas. El estimador equivale al de N/p puntos, con error típico $\sqrt{p}$ veces mayor ($1,04 \cdot 10^{-3}$ en lugar de $5,2 \cdot 10^{-4}$; el observado fue $1,70 \cdot 10^{-3}$). El error reportado suponiendo N muestras subestima la incertidumbre real en un factor $\sqrt{p}$. Además, aumentar p *empeoraría* la precisión.

Una variante más sutil es la del apartado anterior: semillas distintas pero arbitrarias. Puede haber solapamiento parcial (5,2 % de muestras duplicadas con $p = 8$ en nuestro caso).

II.4. Atributos de compartición de datos

En la región `#pragma omp parallel for schedule(static) reduction(+:dentro)` de `montecarlo.c`:

Variable	Atributo	Por qué
`N`, `semilla`, `bloques`	**Compartida** (regla por defecto: declarada fuera de la región)	Solo lectura; `semilla` y `bloques` además son `const`.
`hilos_reales`	**Compartida**	La escribe **una sola** iteración (`b == 0`), así que no hay dos escritores. Se lee después de la barrera implícita al final del `parallel for`, que garantiza la visibilidad.
`dentro`	**Reducción** (`reduction(+:dentro)`)	Ver abajo.
`b`	**Privada**	La variable de iteración del ciclo asociado a un `for` de OpenMP es privada por regla.
`estado`, `aciertos`, `i`, `x`, `y`	**Privadas** (locales)	Son variables automáticas de `muestrear_bloque`, en la pila de cada hilo.

Diferencia entre privada y local: una variable declarada **fuera** de la región puede hacerse privada con `private(v)` (cada hilo recibe una copia **sin inicializar**) o `firstprivate(v)` (copia inicializada con el valor original). Una variable declarada **dentro** de la región o de una función llamada desde ella es local y, por tanto, privada automáticamente. Nuestro código usa este segundo mecanismo, que es más claro: todo lo que debe ser privado se declara donde se usa.

Cláusula usada para el acumulador: `reduction(+:dentro)`. Cada hilo recibe una copia privada inicializada en 0 y acumula en ella. Al final, OpenMP suma las copias sobre `dentro`. No se necesita sincronización dentro del ciclo. Alternativas: `#pragma omp atomic` en cada incremento sería correcto pero haría $\sim 7{,}9 \cdot 10^7$ operaciones atómicas compitiendo por la misma línea de caché; `critical` sería aún peor.

Sin esa cláusula (`./defectos sin_reduccion 10000000 4`, contador compartido con `dentro++`):

Ejecución	dentro	$\hat{\pi}$
1	2 807 383	1,122953
2	2 273 006	0,909202
3	2 667 752	1,067101
4	2 209 966	0,883986
5	2 566 551	1,026620
Correcto	7 856 330	3,142532

Diagnóstico: es una **condición de carrera (data race)** sobre una variable compartida. `dentro++` no es atómico: se traduce en leer, sumar y escribir. Si dos hilos leen el mismo valor, ambos escriben valor+1 y se pierde un incremento. Según el modelo de memoria de C11/OpenMP una carrera es comportamiento indefinido: el compilador puede, por ejemplo, mantener `dentro` en un registro y escribirlo de vez en cuando, lo que pierde bloques enteros de incrementos. **Efecto:** se perdió entre el 64 % y el 72 % de los aciertos. El resultado cambia en cada ejecución y está **sesgado siempre hacia abajo** (solo se pierden aciertos, nunca se ganan), así que $\hat{\pi}$ subestima π. Este error no es estadístico: no se corrige aumentando N.

Las pruebas automáticas usan este caso como control: verifican que la versión correcta da el mismo `dentro` en 10 ejecuciones con $p = 8$ y que la versión defectuosa **sí** se detecta (`tests/run_tests.sh`, prueba 8).

II.5 (opcional). `schedule(static)` frente a `schedule(dynamic)`

Configuración: $N = 10^8$, semilla 20261018, mismo generador; $p = 4$ y $p = 8$; 11 ejecuciones por configuración, intercaladas, más una de calentamiento no registrada (`make schedule`, datos en

`resultados/tiempos_schedule.csv`, tabla en `resultados/tabla_schedule.md`). La unidad de reparto es un bloque de 65 536 muestras; `dynamic` usa chunk = 1 bloque (1 526 bloques).

p	Política	Mediana [s]	Mín – Máx [s]
4	static	0,2376	0,2311 – 0,2797
4	dynamic	0,2330	0,2282 – 0,2423
8	static	0,2446	0,2309 – 0,2568
8	dynamic	0,2334	0,2296 – 0,2403

Las dos políticas dieron el mismo `dentro` en todas las ejecuciones.

Análisis:

- **Con $p = 4$ no hay un efecto observable.** La diferencia de medianas (−2,0 % para dynamic) es menor que la dispersión de cada configuración: static tiene una ejecución atípica de 0,280 s, y sus rangos se solapan casi por completo.
- **Por qué `static` tiene menos sobrecarga en teoría:** con `static` el reparto se calcula una vez (cada hilo sabe su rango sin comunicarse). Con `dynamic` cada hilo, al terminar un chunk, pide el siguiente a un contador compartido (una operación atómica), y esa sobrecarga crece con el número de chunks. Como el trabajo por muestra es prácticamente uniforme, el balanceo que ofrece `dynamic` no aporta nada en una máquina dedicada. En nuestra implementación esa sobrecarga es invisible: cada chunk representa unos 0,57 ms de cómputo ($65\,536 \times 8{,}7$ ns), frente a una petición atómica del orden de 0,1–1 µs, es decir, menos de 0,2 % del tiempo. Si el chunk fuera de **una** muestra (8,7 ns), la sobrecarga dominaría; nuestro diseño por bloques evita ese caso a propósito.
- **Con $p = 8$** (sobresuscripción: 8 hilos en 4 núcleos), `dynamic` fue un 4,6 % más rápido y con menos dispersión. **Hipótesis (no verificada):** con `static` cada uno de los 8 hilos debe terminar su octavo fijo mientras el sistema operativo los alterna en 4 núcleos, y el más retrasado determina el tiempo total. Con `dynamic` los hilos que reciben más tiempo de CPU toman más bloques y compensan. Confirmarlo exigiría trazas del planificador (por ejemplo, `perf sched`), que no hicimos.

Conclusión: para este problema, con tantos hilos como núcleos, la política de planificación no tiene un efecto medible; preferimos `static` por ser más simple y determinista. `dynamic` solo mostró una ligera ventaja al usar más hilos que núcleos, configuración que de todos modos no conviene usar.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl (30 %)

Todos los valores de esta sección salen de mediciones reales (`resultados/tiempos_speedup.csv`) y fueron calculados por `scripts/analisis.py` (`resultados/analisis.json`, `resultados/amdahl.md`).

Equipo y protocolo

Elemento	Valor (de `resultados/equipo.txt`)
Sistema operativo	Ubuntu 24.04.5 LTS, kernel Linux 6.18 (máquina virtual KVM en la nube)
CPU	Intel Xeon Processor @ 2,10 GHz (modelo 207), 1 socket
Núcleos	4 CPU lógicas = 4 núcleos × 1 hilo por núcleo según `lscpu` (son vCPU: no sabemos cómo se asignan a núcleos físicos del anfitrión)
Caché	L1d 48 KiB/núcleo, L2 2 MiB/núcleo, L3 260 MiB compartida
Memoria	15 GiB
Compilador	gcc 13.3.0, `-O2 -fopenmp`, OpenMP 4.5 (`_OPENMP 201511`), glibc 2.39
Programa	`montecarlo.c`, `schedule(static)`, semilla 20261018

Protocolo: $N = 10^8$ fijo; $p \in \{1, 2, 4, 8\}$; 7 ejecuciones por configuración (mínimo pedido: 5), más una de calentamiento no registrada. Las ejecuciones se **intercalan** (una de cada configuración por ronda, con el orden rotado) para que una perturbación pasajera de la máquina no afecte solo a una configuración. Se

registran todos los tiempos individuales y se usa la **mediana**. Todas las ejecuciones dieron `dentro = 78 540 833` (verificado por el script).

¿Por qué la mediana y no el promedio? En un sistema compartido, el ruido (otros procesos, interrupciones, migración de hilos, la máquina virtual) solo puede **sumar** tiempo: hay un mínimo físico y una cola de valores atípicos hacia arriba. El promedio se desplaza con cada valor atípico, mientras que la mediana no. Ejemplo real: en el experimento de II.5, con $p = 4$ y `static`, una ejecución tardó 0,280 s frente a una mediana de 0,238 s; esa sola medición sube el promedio sin representar el comportamiento típico del programa.

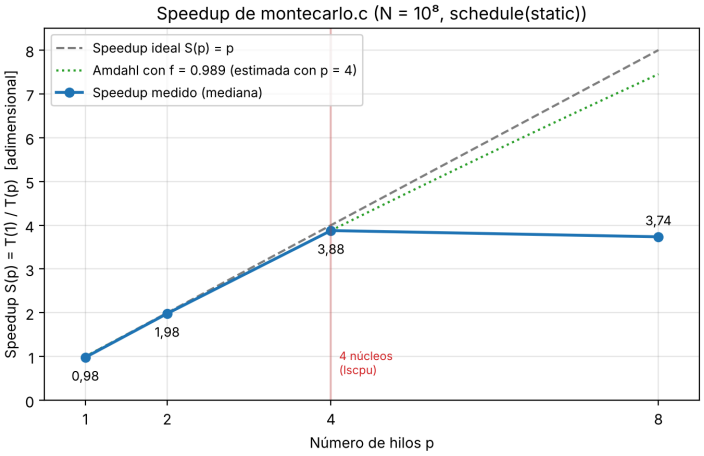
¿Qué es $T(1)$? Usamos la mediana de la **versión secuencial real** (`montecarlo_secuencial`, mismo N, mismo generador, mismo ciclo y la misma forma de medir con `omp_get_wtime()`), como pide II.1. Así el speedup mide la ganancia real frente al mejor programa secuencial disponible. También medimos `montecarlo` con $p = 1$: es un 2,0 % más lento (0,8908 frente a 0,8735 s) por la sobrecarga de la región paralela y el recorrido por bloques. Esa diferencia es del orden de la dispersión de las mediciones (0,860–0,896 s frente a 0,860–0,915 s). Usar la versión OpenMP con $p = 1$ como $T(1)$ inflaría un poco el speedup (columna “S vs OpenMP p=1” en `resultados/tabla_speedup.csv`).

Tabla experimental

$N = 10^8$, semilla 20261018, `schedule(static)`. π estimado = 3,14163332 en **todas** las filas (error absoluto $4,07 \cdot 10^{-5}$).

p **(hilos)**	**Tiempos individuales [s]**	**Mediana de $T(p)$ [s]**	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1 (secuencial)	0,8888 · 0,8735 · 0,8601 · 0,8720 · 0,8648 · 0,8963 · 0,8865	**0,8735**	1,000	1,000
1 (OpenMP)	0,8912 · 0,8855 · 0,8908 · 0,8603 · 0,9094 · 0,8675 · 0,9152	**0,8908**	0,981	0,981
2	0,4590 · 0,4612 · 0,4408 · 0,4246 · 0,4601 · 0,4404 · 0,4291	**0,4408**	1,982	0,991
4	0,2345 · 0,2306 · 0,2276 · 0,2228 · 0,2253 · 0,2155 · 0,2150	**0,2253**	3,877	0,969
8	0,2483 · 0,2411 · 0,2275 · 0,2338 · 0,2338 · 0,2344 · 0,2263	**0,2338**	3,735	0,467

Gráfica: `resultados/speedup.png` (speedup medido, recta ideal $S = p$ y la curva de Amdahl con la f estimada).



Speedup

III.1. Ley de Amdahl en tres casos

$$S(p) = \frac{1}{(1-f) + f/p}$$

(a) $f = 0,98$**,** $p = 8$**:** $S = \frac{1}{0,02 + 0,98/8} = \frac{1}{0,02 + 0,1225} = \frac{1}{0,1425} = \mathbf{7,02}$ → eficiencia $7,02/8 = 0,877$. Con solo un 2 % de código secuencial ya se pierde el 12 % de la máquina con 8 procesadores.

(b) $f = 0,90$**,** $p = 8$**:** $S = \frac{1}{0,10 + 0,90/8} = \frac{1}{0,10 + 0,1125} = \frac{1}{0,2125} = \mathbf{4,71}$ → eficiencia 0,588. Pasar de 2 % a 10 % de parte secuencial reduce el speedup de 7,0 a 4,7: con 8 hilos, la parte secuencial (0,10) ya tarda casi lo mismo que toda la parte paralela (0,1125).

(c) $f = 0,98$**,** $p \to \infty$**:** $S_\infty = \lim_{p \to \infty} \frac{1}{0,02 + 0,98/p} = \frac{1}{0,02} = \mathbf{50}$.

Lección del caso (c): el speedup máximo está acotado por $1/(1-f)$, **sin importar cuántos procesadores se agreguen**. Con infinitos procesadores la parte paralela tarda cero, pero la secuencial sigue tardando lo mismo. Un 2 % de código secuencial, que parece insignificante, limita el programa a 50× para siempre. Por eso, para escalar a muchos procesadores, reducir la fracción secuencial importa más que agregar hardware.

III.2. Estimación de f con el speedup medido para $p = 4$

Despeje:

$$S = \frac{1}{(1-f) + f/p} \;\Rightarrow\; \frac{1}{S} = 1 - f + \frac{f}{p} = 1 - f\left(1 - \frac{1}{p}\right) \;\Rightarrow\; f = \frac{1 - 1/S}{1 - 1/p}$$

Sustitución con $S(4) = 0,8735/0,2253 = 3,877$:

$$f = \frac{1 - 1/3,877}{1 - 1/4} = \frac{1 - 0,2579}{0,75} = \frac{0,7421}{0,75} = \mathbf{0,9895} \qquad \Rightarrow \qquad 1 - f = \mathbf{0,0105}\ (1,05\,\%).$$

El valor está en $[0, 1]$ y es físicamente interpretable: según Amdahl, el 98,9 % del trabajo se comporta como paralelizable y el 1,05 % como secuencial.

Métrica de Karp-Flatt:

$$e(p) = \frac{1/S(p) - 1/p}{1 - 1/p}$$

p	$S(p)$	$e(p)$
2	1,982	0,0092
4	3,877	0,0105
8	3,735	0,1631

Con un solo p, e coincide algebraicamente con $1 - f$ (0,0105 con $p = 4$). Su utilidad está en **ver cómo cambia con** p. Si e fuera constante, la pérdida se debería a una fracción secuencial fija (Amdahl puro). Si e crece con p, hay una **sobrecarga paralela** que aumenta con el número de hilos (sincronización,

comunicación, contención o falta de núcleos), algo que Amdahl no modela. Nuestros datos muestran $e \approx 0,01$ casi constante para $p = 2$ y 4, y un salto a 0,16 con $p = 8$. Ese salto no es una fracción secuencial: es la falta de núcleos (ver III.3).

¿Qué es realmente ese 1 %? En la región medida, lo estrictamente secuencial (crear el equipo de hilos, repartir 1 526 bloques, la reducción de 4 valores, la barrera final) cuesta del orden de microsegundos, mientras que el 1,05 % de 0,87 s son unos 9 ms. Además, el desbalance de carga de `static` es mínimo (382 frente a 381 bloques por hilo, ≈ 0,13 %). Por tanto, la "fracción secuencial" estimada **no es código secuencial**: recoge otros efectos que Amdahl no distingue, como la sobrecarga de la versión OpenMP frente a la secuencial (+2,0 % con $p = 1$) y la variación del equipo. Si se toma como referencia la versión OpenMP con $p = 1$, el speedup con 4 hilos es 3,954 y $e(4) = 0,0039$, menos de la mitad.

III.3. Speedup ideal, medido y predicho por Amdahl ($f = 0,9895$)

p	Ideal $S = p$	Medido	Amdahl con $f = 0,9895$	Medido / Amdahl
1	1	0,981	1,000	0,98
2	2	1,982	1,979	1,00
4	4	3,877	3,877 (ajustado aquí)	1,00
8	8	3,735	7,450	0,50

Hasta 4 hilos el programa escala casi idealmente (eficiencia 0,99 con $p = 2$ y 0,97 con $p = 4$) y Amdahl describe bien los datos. **Con 8 hilos la discrepancia es enorme** (3,7 frente a 7,5).

Causas **observadas** (respaldadas por nuestras mediciones):

1. **Disponibilidad real de núcleos.** La máquina tiene 4 CPU (`nproc = 4`). Con 8 hilos, el sistema operativo alterna dos hilos por núcleo: el trabajo total es el mismo y la capacidad de cómputo también, así que $T(8) \approx T(4)$ (0,2338 frente a 0,2253 s). Amdahl supone p procesadores reales; con $p >$ núcleos, su predicción no aplica. El 3,8 % adicional de $T(8)$ es el costo de los cambios de contexto y de que el más lento de los 8 hilos determina el final.
2. **Sobrecarga de la versión paralela con un hilo:** +2,0 % (0,8908 frente a 0,8735 s), del orden de la dispersión.
3. **Ruido experimental:** los tiempos individuales con $p = 4$ van de 0,2150 a 0,2345 s (±4 %). Con el mínimo, el speedup daría 4,06, por encima del ideal, lo cual es imposible y confirma que hay ruido. Por esto se usa la mediana. Antes de esta implementación, con la variante SplitMix64 (`src/variantes/`) en la misma máquina virtual, tres series con $N = 10^8$ dieron $S(4)$ entre 2,9 y 3,96 (ver bitácora): la VM comparte el anfitrión con otros procesos.

Factores **descartados o despreciables**, con su razón:

4. **Creación y destrucción de hilos:** libgomp crea los hilos una vez y los reutiliza; el costo es de decenas de microsegundos frente a 225 ms (< 0,05 %).
5. **Costo de la reducción:** p sumas de enteros al final: despreciable.
6. **Generación de números aleatorios:** cada hilo tiene su estado en un registro; no hay contención. El salto inicial por bloque (1 526 saltos de ~30 multiplicaciones) es despreciable. En cambio, *compartir* el generador sí destruiría el speedup (II.3: hasta 18× más lento).
7. **Ancho de banda de memoria y jerarquía de caché:** el cálculo no lee ni escribe arreglos. Todo cabe en registros y en L1, así que no hay tráfico de memoria que saturar. Es un problema limitado por cómputo, y por eso escala tan bien hasta 4 hilos. Las copias privadas de la reducción viven en la pila de cada hilo, de modo que tampoco hay *false sharing* en el ciclo.

Hipótesis plausibles no verificadas:

8. **Frecuencia variable del procesador:** muchos Xeon bajan la frecuencia turbo cuando hay más núcleos activos, lo que reduciría la eficiencia al pasar de 1 a 4 núcleos (podría explicar parte del 3 % que se pierde con $p = 4$). En la VM no tenemos acceso a la frecuencia real (`lscpu` no informa MHz dinámicos), así que no se pudo verificar.

9. **Hiperprocesamiento:** `lscpu` informa 1 hilo por núcleo, pero en una VM las 4 vCPU podrían estar asignadas a hilos hermanos de un mismo núcleo físico del anfitrión. Si fuera así, el speedup con 4 hilos sería claramente menor que 4; como obtuvimos 3,88, todo indica que son núcleos reales, aunque no podemos asegurarlo.

III.4. Proyección a una máquina de 64 núcleos

Con $f = 0,9895$:

$$S(64) = \frac{1}{0,0105 + 0,9895/64} = \frac{1}{0,0105 + 0,01546} = \frac{1}{0,0260} = \mathbf{38,5} \qquad E(64) = 0,60 \qquad S_\infty = \frac{1}{0,0105} = 94,8.$$

Con N fijo, 64 núcleos darían como máximo ≈ 38× (60 % de eficiencia). Con 64 hilos, la parte "secuencial" (1,05 %) ya tarda casi lo mismo que la paralela (1,55 %).

Sensibilidad: la proyección es muy sensible al $S(4)$ medido, porque extrapola de 4 a 64:

$S(4)$ **medido**	f	$S(64)$	S_∞
3,750	0,9778	26,7	45
3,877 (nuestro)	**0,9895**	**38,5**	**95**
3,950	0,9958	50,6	237

Una diferencia de 5 % en $S(4)$ cambia la predicción para 64 núcleos casi al doble. Lo correcto sería medir de nuevo en la máquina de 64 núcleos (con varios sockets, NUMA o hiperprocesamiento, los efectos pueden ser otros).

¿En qué se diferencia de crecer N? Amdahl supone N **fijo**. En Monte Carlo el trabajo paralelo crece con N, mientras que la parte que no escala es casi constante. De $T(1)$ y $T(4)$:

$$T(1) = t_s + t_{par}, \quad T(4) = t_s + t_{par}/4 \;\Rightarrow\; t_s = \frac{4T(4) - T(1)}{3} = 9,2 \text{ ms}, \quad t_{par} = 864 \text{ ms}.$$

Si N se multiplica por k, t_{par} se multiplica por k y t_s no, así que f crece:

N	f	$S(64)$
10^8	0,9895	38,5
10^9 (×10)	0,9989	60,0
10^{10} (×100)	0,99989	63,6

Aumentar el problema hace que la máquina grande se aproveche mejor (ver III.5).

Reflexión: ¿p mejora la precisión o solo el tiempo? Con N fijo, aumentar p **solo mejora el tiempo**. Como se demostró en I.3, el estimador es el mismo; en nuestra implementación incluso el resultado es idéntico bit a bit (`dentro = 78 540 833` para todo p). El error típico depende únicamente de N: $\sigma = 1,64/\sqrt{N}$.

- p **mejora el tiempo:** el mismo cálculo termina hasta $S(p)$ veces antes.
- N **mejora la precisión:** el error típico baja como $1/\sqrt{N}$, a costa de un tiempo que crece linealmente con N.

Combinadas, los núcleos se pueden usar para "comprar" precisión: en el mismo tiempo de pared que la versión secuencial con $N = 10^8$ (0,87 s), 64 núcleos procesarían del orden de $38,5 \times 10^8$ muestras (más, porque f crece con N), reduciendo el error típico unas $\sqrt{38,5} \approx 6,2$ veces (hasta $\sqrt{64} = 8$ veces en el mejor caso). Ni siquiera así se gana un dígito completo, que requiere 100 veces más muestras. **Limitación de nuestra implementación:** el flujo de `rand_r` admite como máximo $N = 715\,827\,882$ muestras sin repetirse (II.2), así que para $N \geq 10^9$ habría que cambiar a un generador de período mayor.

III.5 (opcional). Por qué Amdahl puede ser pesimista: Gustafson

Amdahl responde a la pregunta *"¿cuánto más rápido resuelvo **el mismo problema**?"* (escalado fuerte). En la práctica, con más procesadores se suele resolver **un problema más grande en el mismo tiempo** (escalado débil): en Monte Carlo, más muestras para mayor precisión.

Ley de Gustafson. Sea α la fracción del tiempo **de la ejecución paralela** que es secuencial. Si la parte paralela se ejecutara en un solo procesador, tardaría p veces más:

$$S_G(p) = \frac{\alpha + p(1-\alpha)}{\alpha + (1-\alpha)} = \alpha + p\,(1-\alpha) = p - \alpha\,(p-1).$$

	Amdahl	**Gustafson**
Qué se fija	El tamaño del problema (N)	El tiempo de ejecución
Fracción serial	$1-f$, medida sobre el tiempo **secuencial**	α, medida sobre el tiempo **paralelo**
Al crecer p	La parte serial pesa cada vez más → techo $1/(1-f)$	El trabajo paralelo crece con $p \to S$ crece linealmente, sin techo

Con nuestros datos: $\alpha = t_s/T(4) = 9,2/225,3 = 0,041$ y $S_G(64) = 0,041 + 64 \times 0,959 =$ **61,4**, frente a 38,5 de Amdahl con N fijo. Ambas leyes son correctas, pero responden a preguntas distintas. Amdahl es pesimista solo si en realidad vamos a aprovechar los procesadores adicionales para hacer más trabajo, que es justo lo que conviene en Monte Carlo. La tabla de III.4 ($S(64) \to 63,6$ con $N \times 100$) muestra el mismo fenómeno desde el punto de vista de Amdahl.